

In this example, the original variable *v\_commission* anchors *v\_total\_commission*, which in turn is used to anchor *v\_net\_commission*. There is no limit on the number of layers of nesting in anchored declarations.

The source variable for a %TYPE declaration does not have to be in the same block. The variable could be a global variable declared at the SQL\*Plus environment, or it could be declared in a block that contains the current block.

### NOT NULL Constraint for %TYPE Declarations

If a source variable is declared with a NOT NULL constraint, the %TYPE declaration inherits the NOT NULL constraint from the source, its anchor. The anchored variable must be initialized with a value in the %TYPE declaration.

If the source for a %TYPE declaration is a table's column, the NOT NULL constraint is not inherited by the anchored variable. There is no need to initialize the anchored variable, and it can be assigned a NULL value.

## ASSIGNMENT OPERATION

The assignment operation is one of the ways to assign a value to a variable. You have already learned that a variable can be initialized at the time of declaration by using the DEFAULT option or :=. The assignment operation is used in the executable section of the program block to assign a literal, another variable's value, or the result of an expression to a variable. The general syntax is

*VariableName* := *Literal* | *VariableName* | *Expression*;

For example,

```
v_num1      := 100;
v_num2      := v_num1;
v_sum       := v_num1 + v_num2;
```

In these examples, the assumption is made that three variables have already been declared. The first example assigns 100 to the variable *v\_num1*. The second example assigns the value of the variable *v\_num1* to the variable *v\_num2*. The third example assigns the result of an addition operation on *v\_num1* and *v\_num2* to the variable *v\_sum*.

The following statements are examples of invalid assignment operations and the reasons for their lack of validity:

```
v_count = 10;           /* Wrong assignment operator, = sign */
v_count * 2 := v_double; /* Expression cannot be on the left. */
v_num1 := v_num2 := v_num3; /* Cannot use two assignments in one statement. */
```

## BIND VARIABLES

Bind variables are also known as host variables. These variables are declared in the host SQL\*Plus environment and are accessed by a PL/SQL block. Anonymous blocks do not take any arguments, but they can access host variables with a colon prefix (:) and the host variable name. Host variables can be passed to procedures and functions as arguments. A host variable is declared at the SQL> prompt with the SQL\*Plus VARIABLE statement. The syntax of a host variable declaration is

**VARIABLE *variablename datatype***

For example,

**SQL> VARIABLE double NUMBER**

When a numeric variable is declared with VARIABLE command, precision and scale values are not used. If a VARCHAR2-type variable is declared, length is not used. A host variable's value can be printed in the SQL\*Plus environment by using the PRINT command.

Let us put everything together in a program. The program contains a script that includes SQL\*Plus statements and a PL/SQL block.

In Figure 10-5, two types of variables are used, a local variable *v\_num* and a host variable *g\_double*. The host variable *g\_double* is declared in SQL\*Plus with a VARIABLE statement, and the program block references it with a colon prefix (:). The local variable *v\_num* is declared in the declaration section of a program block; there is no need to use the colon prefix with it. The program assigns the value 5 to

```
SQL> VARIABLE g_double NUMBER
SQL> DECLARE
  2     v_num  NUMBER(2);
  3 BEGIN
  4     v_num := 5;
  5     :g_double := v_num * 2;
  6 END;
  7 /

PL/SQL procedure successfully completed.

SQL> PRINT g_double

  G_DOUBLE
-----
          10

SQL>
```

**Figure 10-5** Using a host variable in a PL/SQL block.

the local variable *v\_num*, doubles it, and stores the result in the host variable *g\_double*. Finally, the resulting variable is printed in the host environment with a PRINT statement.

**Question:** How does a PL/SQL block end?

**Answer:** It ends with an END and a semicolon on the same line *and* a slash (/) on the next line.

The use of host variables should be minimized in a program block, because they affect performance. Every time a host variable is accessed within the block, the PL/SQL engine must stop to request the host environment for the value of the host variable. The variable's value can be assigned to a local variable to minimize calls to the host.

## SUBSTITUTION VARIABLES IN PL/SQL

PL/SQL does not have any input capabilities in terms of having an input statement. There are no explicit input/output (I/O) statements, but substitution variables of SQL are available in PL/SQL. Substitution variables have limitations, which become apparent in a loop.

Let us rewrite the program code in Figure 10-5 to the one in Figure 10-6. When the code in Figure 10-6 is executed, a standard prompt for *p\_num* appears on the screen for users to type in a value for it. As you see in the example, there is no need

```
SQL> VARIABLE g_double NUMBER
SQL> DECLARE
  2     v_num  NUMBER(2);
  3 BEGIN
  4     v_num := &p_num;
  5     :g_double := v_num * 2;
  6 END;
  7 /
Enter value for p_num: 10

PL/SQL procedure successfully completed.

SQL> PRINT g_double

   G_DOUBLE
-----
          20

SQL>
```

**Figure 10-6** Local, host, and substitution variables.

to declare substitution variables in the program block. The value of the bind/host variable is printed with the PRINT command. The value of the local variable *v\_num* cannot be printed with the PRINT command, however, because the scope of a local variable is within the block where it is declared.

When substitution variable is used in a program, the output contains lines showing how the substitution was done. You can suppress those lines by using the SET VERIFY OFF command before running the program.

## PRINTING IN PL/SQL

There is no explicit output statement in PL/SQL. Oracle does have a built-in package called DBMS\_OUTPUT with the procedure PUT\_LINE to print. An environment variable named SERVEROUTPUT must be toggled ON to view output from it.

The DBMS\_OUTPUT is the most frequently used package because of its capabilities to get lines from a file and to put lines into the buffer. The PUT\_LINE procedure displays information passed to the buffer and puts a new-line marker at the end. For example,

```
DBMS_OUTPUT.PUT_LINE ('This line will be displayed');
```

The maximum size of the buffer is 1 megabyte. The following command enables you to view information from the buffer by using the DBMS\_OUTPUT package and also sets the buffer size to the number of bytes specified:

```
SET SERVEROUTPUT ON [on size 10000];
```

The PL/SQL block in Figure 10-7 shows the use of DBMS\_OUTPUT.PUT\_LINE. Another procedure, DBMS\_OUTPUT.PUT, also performs the same task of

```
SQL> VARIABLE NUM NUMBER
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2   DOUBLE NUMBER;
  3 BEGIN
  4   :NUM := 5;
  5   DOUBLE := :NUM * 2;
  6   DBMS_OUTPUT.PUT_LINE (('DOUBLE OF ' ||
  7   TO_CHAR(:NUM) || ' IS ' || TO_CHAR(DOUBLE));
  8 END;
  9 /
DOUBLE OF 5 IS 10

PL/SQL procedure successfully completed.

SQL>
```

**Figure 10-7** DBMS\_OUTPUT.PUT\_LINE.

displaying information from the buffer, but it does not put a new-line marker at the end. If there is another `DBMS_OUTPUT.PUT` or `DBMS_OUTPUT.PUT_LINE` statement following that statement, its output will be displayed on the same line.

## ARITHMETIC OPERATORS

Five standard arithmetic operators are available in PL/SQL for calculations. If more than one operator exists in an arithmetic expression, the following order of precedence is used:

- Exponentiation is performed first, multiplication and division are performed next, and addition and subtraction are performed last.
- If more than one operator of the same priority is present, they are performed from left to right.
- Whatever is in parentheses is performed first.

Figure 10-8 shows arithmetic operators and their use.

| Arithmetic Operator | Use                      |
|---------------------|--------------------------|
| +                   | Addition                 |
| −                   | Subtraction and negation |
| *                   | Multiplication           |
| /                   | Division                 |
| **                  | Exponentiation           |

**Figure 10-8** Arithmetic operators.

**Question:** What is the answer from the following expression?

$$-2 + 3 * (10 - 2 * 3)/6$$

**Answer:** 0 (The operations within the parentheses are performed first, with multiplication followed by subtraction. The result from within the parentheses is multiplied by 3, and that result is divided by 6. Finally, the result is added to −2)

In this chapter, you learned the basics of PL/SQL. In the next chapter, you will learn about three programming control structures: sequence, selection, and looping. (You already know one of them. All examples in this chapter were based on sequence

structure.) You will also learn to interface with the Oracle server by embedding SQL statements in PL/SQL program blocks.

## IN A NUTSHELL . . .

- PL/SQL (Programming Language extension to Structured Query Language) is Oracle's proprietary language.
- PL/SQL contains features of modern languages, such as data encapsulation, error handling, information hiding, and object-oriented programming (OOP).
- PL/SQL is a block-structured language. A block is of two types: anonymous block and subprogram (procedures and functions).
- A program code contains reserved words, user-defined identifiers, delimiters, comments, and literals.
- A program block consists of three sections: declaration, executable, and exception handling.
- A program uses variables and constants to hold values. A variable's value can be changed, but a constant's value remains the same throughout the execution of the program.
- A variable is declared in the declarative section with a scalar data type. The standard data types are number, character, Boolean, and date. PL/SQL also supports other LOB (Large Object) data types.
- A declaration attribute %TYPE is used to anchor a variable with another variable's data type or with a table column's data type.
- A bind variable, or host variable, is global to a PL/SQL block. An anonymous block refers to it with a colon prefix (:). A bind variable is declared with the VARIABLE statement and is printed with the PRINT statement in the SQL\*Plus environment.
- An assignment statement is used in the executable section to assign a literal, a variable's value, or the result of an expression to a variable. An assignment uses the := operator.
- PL/SQL does not have an input statement, but substitution variables are allowed in a block to assign a value to a variable.
- A built-in Oracle package and its procedure DBMS\_OUTPUT.PUT\_LINE are used to output information. An environment variable SERVEROUTPUT must be set to ON before using it.
- Arithmetic operators (+, -, \*, /, and \*\*) are used in mathematical expressions. The operations follow rules of precedence for evaluating expressions with more than one operator.

## EXERCISE QUESTIONS

### True/False:

1. PL/SQL is a nonprocedural language developed by Oracle Corporation.
2. The SQL language has built-in error checking and error handling, but PL/SQL does not.
3. Three types of variables used in a PL/SQL program are local, host, and substitution.
4. A variable with a NOT NULL constraint and a constant must be initialized with a value at the declaration time.
5. An assignment statement is used in the executable section to assign value to a variable or a constant.
6. In a declaration with the %TYPE attribute, the source is either a variable or a column in a table.
7. If the source variable is declared with a NOT NULL constraint, the anchored variable inherits the same constraint.
8. If the source column in a table has a NOT NULL constraint, the anchored variable also gets the NOT NULL constraint.
9. Exponentiation is performed before addition in an expression without any parentheses.
10. The declaration and executable sections are mandatory in a PL/SQL block.
11. A bind variable is declared with a VARIABLE command at the SQL> prompt.
12. A PL/SQL block must contain BEGIN and END key words.
13. A substitution variable is declared under the DECLARE section of a PL/SQL block.
14. A bind variable is used in a PL/SQL block with a colon prefix (:).
15. A local variable used in a PL/SQL block can be printed with the PRINT command.

### Answer the Following Questions:

1. State differences between Oracle's SQL and PL/SQL languages.
2. What are the two types of blocks in PL/SQL? What are the differences between them?
3. Name four standard scalar data types used in PL/SQL. When is each type used for variables?
4. Name three types of variables used in PL/SQL. Where are they declared? Give a sample declaration of each.
5. Give any differences and similarities between := and DEFAULT.
6. What are advantages of the %TYPE attribute in a variable declaration?
7. State the rules of precedence used in arithmetic operations.
8. How will you declare a bind variable named *v\_count*, use it in a PL/SQL block, and print its value?
9. Declare a variable *v\_val*, which may not have a null value. Assign a value to *v\_val* with a substitution variable, and print that value.
10. How do you run a PL/SQL block?

**LAB ACTIVITY**

1. Create a program script that uses a PL/SQL anonymous block to perform the following: Use a host variable AREA to store the result. Declare a local variable RADIUS with numeric data type. Declare a constant PI with value 3.14. Assign a value to the variable RADIUS by using a substitution variable. Calculate area of a circle by using the formula

$$\text{AREA} = \text{PI} * \text{RADIUS} * \text{RADIUS}$$

Then, print the result in SQL \* Plus.

2. Write a PL/SQL block to find the square, cube, and double of a number inputted with a substitution variable, and print the results using the built-in package DBMS\_OUTPUT.
3. Write a PL/SQL block to swap the values of two variables. Print the variables before and after swapping.
4. Write a PL/SQL program to input hours and rate. Find gross pay and net pay with a tax rate is 28%. Print your results. (No need to perform overtime calculations.)
5. Write a PL/SQL program with two variables, for the first name and the last name. Print the full name with the last name and first name separated by comma and a space.



# 11

## ***More on PL/SQL: Control Structures and Embedded SQL***

### **IN THIS CHAPTER . . .**

- You will learn about various programming control structures in PL/SQL.
- Different decision-making statements based on various options are covered.
- Looping statements are introduced to perform a set of statements repetitively.
- SQL statements are embedded within a PL/SQL block to interact with the Oracle server.

In the previous chapter, you learned the basics of the PL/SQL programming language. You are now able to write simple programs using local, host, and substitution variables; can perform simple calculations by using assignment statements; and know how to use Oracle's built-in package DBMS\_OUTPUT.PUT\_LINE in program blocks to display results from the buffer. The sample programs and lab exercises have a series of statements that are executed from the beginning to the end in a linear fashion. When an anonymous block is executed, the code is sent to the PL/SQL engine for compilation. In this chapter, you will see the use of different control structures employed in a high-level programming language.

In the last chapter, you saw how to write PL/SQL programs independent of a database. We start with more independent PL/SQL programs, and then show the actual use of PL/SQL to interact with the Oracle database. A PL/SQL program block “talks” to the Oracle database by embedding SQL statements in its executable section.

## CONTROL STRUCTURES

In a procedural language like PL/SQL, there are three basic programming control structures:

1. In a **sequential structure**, a series of instructions are performed from the beginning to the end in a linear order. None of the instructions is skipped, and none of the instructions is repeated.
2. The **selection structure** is also known as a **decision structure** or an **IF-structure**. It involves conditions with a TRUE or FALSE outcome. Based on the outcome, one of the options is performed, and the other option is skipped. Selection statements are also available for multiple options.
3. In a **looping structure**, a series of instructions is performed repeatedly. There are different looping statements appropriate for a variety of situations. A programmer has to write a loop correctly to make it perform a specific number of times.

We have already covered sequential statements in the previous chapter. In this chapter, we will talk about the selection and the looping structures. In actuality, a program may utilize one or a combination of all control structures.

### Selection Structure

There are three selection or conditional statements in PL/SQL. Relational operators, logical operators, and other special operators are used to create Boolean expressions or conditions. The tables in Figures 11-1, 11-2, and 11-3 are repeated from Chapter 5 for reading convenience. Figure 11-1 shows the use of relational operators, which constitute simple conditions. Figure 11-2 explains the use of logical operators in compound conditions. Figure 11-3 shows a truth table for the AND, OR, and NOT operators. The AND and OR operators are binary operators, because they work on two conditions. The NOT operator is a unary operator, because it works on a single condition.

| Relational Operator | Meaning                  |
|---------------------|--------------------------|
| =                   | Equal to                 |
| <> or !=            | Not equal to             |
| >                   | Greater than             |
| >=                  | Greater than or equal to |
| <                   | Less than                |
| <=                  | Less than or equal to    |

**Figure 11-1** Relational operators.